3.1 Data, variables, samples

Vladimir Buskin

English Language and Linguistics, Catholic University of Eichstätt-Ingolstadt

ORCID: 0009-0005-5824-1012

Last updated: 30 September 2026, 19:46

This handout introduces key statistical concepts—such as samples, populations, variables, datasets, and data types—with a focus on their application to empirical linguistic research, using R to explore and illustrate these ideas.

Table of contents

Preparation . . . 2
Recommended reading . . . 2
Samples and populations . . . 2
Population parameters and sample statistics . . . 3
(Random) Variables . . . 3
Datasets . . . 5
Data types . . . 7
Dependent vs. independent variables . . . 8
Loading datasets into R . . . 8
A convenient alternative: RDS files . . . 11

Exercises **12**
Tier 1 . . . 12
Tier 2 . . . 14
Tier 3 . . . 16

References **18**

Preparation

> **Script**
>
> You can find the full R script associated with this unit here.

Recommended reading

Baayen (2008: Chapter 2)

Baguley (2012: Chapter 1)

Agresti & Kateri (2022: Chapter 1.2)

Heumann et al. (2022: Chapters 1 & 7)

Samples and populations

In order to investigate one or more linguistic features, we need to collect relevant **observations**. Actions that generate observations are called **experiments**, such as decision tasks or even corpus queries. Since it is not feasible to examine, for instance, the entirety of all linguistic utterances ever produced, i.e., the virtually infinite **population**, we rely on subsets of it: **samples**. Good research is characterised by good **sampling procedures** that limit the **bias** present in any sample, thus improving the generalisability of potential findings.

To state it more formally, let ω ('lower-case Omega') denote a single observation of interest. The full set of theoretically possible observations is contained in Ω ('upper-case Omega'), with

$$\Omega = \{\omega_1, \omega_2, \dots, \omega_n\}.$$

Here n represents the n-th observation and should be a natural number. A sample is then simply a selection of elements from Ω.

Example 1. A commonly studied phenomenon is the realisation of subject pronouns in English finite clauses. There are two possible outcomes:

- overt subject (e.g., *She went home*)
- null subject (e.g., *Ø went home*)

Then, the virtually infinite population Ω would consist of all subjects ever produced by speakers of English. A sample might contain 3,000 observations, where each observation ω_i represents a single realised subject.

Observation	Subject form
ω_1	overt
ω_2	null
ω_3	overt
ω_4	null
ω_5	overt
⋮	⋮
ω_{3000}	null

Population parameters and sample statistics

Since the true population Ω is generally inaccessible, we cannot directly observe its properties. Instead, we use our sample to compute **sample statistics**, which serve as estimates of the underlying **population parameters**. This estimation problem is one of the central concerns of inferential statistics, which is a topic we will return to in later units (e.g., during hypothesis testing).

For now, it is sufficient to be aware of the distinction: A population parameter is a fixed (but typically unknown) quantity describing the population, while a sample statistic is a quantity we can actually compute from our observed sample (Baguley (2012: 7)). The two are related but not identical. A sample statistic will generally deviate from the true population parameter to some degree (**sampling error**).

Several common parameters and their corresponding statistics are summarised below:

Property	Population parameter	Sample statistics
Size	N	n
Mean	μ	$\hat{\mu}, \bar{x}$
Variance	σ^2	$\hat{\sigma}^2, s^2$
Standard deviation	σ	$\hat{\sigma}, s$
Proportion	π	$\hat{\pi}$
Linear effect	β	$\hat{\beta}$

(Random) Variables

The concept of the **variable** is very handy in that it enables researchers to quantify different aspects of linguistic observations, such as the age or gender of the speaker, the register of the speech situation, the variety of English, the length of an utterance, a syntactic construction or a grammatical category, among many other theoretically possible features.

For each observation, we should be able to assign a specific **outcome** to our variables. While the variable `Genitive` has a small set of outcomes, namely 's' and 'of', other variables such as `Utterance length` or `Reaction time` could technically assume any value in the range

$[0, \infty)$. The set of all possible outcomes in an experiment is known as the **sample space** S.

Example: Sample space of genitives

Given a single observed NP which is marked for possessive case, we may characterise its formal marking as either 's' (synthetic genitive) or as 'of' (analytic/periphrastic). For simplicity, we will call these options S and O. The sample space of `Genitive` could be thus described as

$$S_{\text{Genitive}} = \{\text{S}, \text{O}\}.$$

Experiments normally do not terminate after obtaining the first observation. As a result, the sample space becomes considerably more complex (and unwieldy), the more observations are gathered. For $n = 3$ possessive NPs, the sample space would comprise the following combinations of outcomes:

$$S_{\text{Genitive (3 NPs)}} = \{\text{SSS}, \text{SSO}, \text{SOS}, \text{SOO}, \text{OSS}, \text{OSO}, \text{OOS}, \text{OOO}\}.$$

With $n = 10$ we'd be dealing with $2^{10} = 1024$ possible outcomes. If, for instance, we're only interested in the total number of *s*-genitives, this isn't a particularly efficient approach.

Random variables allow us to map outcomes from the sample space onto the real numbers $\mathbb{R}$. Quite conveniently, we could define a random variable X that counts the total number of *s*-genitives in a sample of size n. Note, however, that the random variable is not random – it is the result of a random process. Other random variables could be

- the number of 'heads' after tossing a coin 30 times,
- getting a '6' when rolling a die 500 times, or
- how many years it takes for your TV to break down.

Random variables: Formal definition and example

Strictly speaking, a random variable is a **function** X which maps onto each outcome $s \in S$ exactly one number $X(s) = x$ with $x \in \mathbb{R}$:

$$X : S \to \mathbb{R}.$$

Recall Example 1, where 3,000 subjects have been observed in one form or another. Rather than tracking every possible sequence, we can define a random variable

$$X_{\text{overt}} : S \to \mathbb{R},$$

which simply counts the total number of overt subjects. If we observe 2,100 overt subjects, then $X_{\text{overt}} = 2100$.

Datasets

Information on variables and their values is conventionally arranged in a **dataset**, which is essentially a matrix with n **rows** (= observations) and p **columns** (= variables). The data matrix has the general form in Equation 1 (see (Heumann, Schomaker, and Shalabh 2022: Chap. 1.4) for details).

$$\begin{pmatrix} \text{Observation } \omega & \text{Variable } X_1 & \text{Variable } X_2 & \cdots & X_p \\ 1 & x_{11} & x_{12} & \cdots & x_{1p} \\ 2 & x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \vdots & & \vdots \\ n & x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix} \tag{1}$$

Each column in our data matrix represents all observed values for a single variable. We can denote this as a column vector:

$$X_1 = \begin{pmatrix} x_{11} \\ x_{21} \\ \vdots \\ x_{n1} \end{pmatrix}$$

Alternatively, using set notation: $X_1 = \{x_{11}, x_{21}, \ldots, x_{n1}\}$.

Example: Data matrix

Example 2. Let's create a small example dataset where the `Genitive` realisation is recorded for different `Speaker`s, including their `Age`:

```
example_data <- data.frame(
  Speaker = c("A", "B", "C", "D"),
  Age = c(25, 34, 29, 41),
  Genitive = c("S", "O", "S", "S")
)

print(example_data)
```

```
  Speaker Age Genitive
1       A  25        S
2       B  34        O
3       C  29        S
4       D  41        S
```

The observation numbers are generally implicit and correspond to the row numbers:

```
rownames(example_data)  # "1" "2" "3" "4"
```

```
[1] "1" "2" "3" "4"
```

We can also make them explicit:

```
library(tidyverse)

# Create a new "Observation" column
example_data$Observation <- 1:nrow(example_data)

# Move column to the front
example_data <- relocate(example_data, Observation)

print(example_data)
```

```
  Observation Speaker Age Genitive
1           1       A  25        S
2           2       B  34        O
3           3       C  29        S
4           4       D  41        S
```

The columns can now be formally re-interpreted as ω, X_1, X_2, and X_3, respectively. Using familiar subsetting operations, we can pick out individual columns and inspect them further:

```
example_data$Speaker # this is X1
```

```
[1] "A" "B" "C" "D"
```

```
length(example_data$Speaker) # X1 has four elements {x1, x2, x3, x4}.
```

```
[1] 4
```

```
# Draw random sample
slice_sample(example_data, n = 2)
```

```
  Observation Speaker Age Genitive
1           1       A  25        S
2           3       C  29        S
```

Data types

In general, we distinguish between **discrete** variables, which can only take a limited set of unique values, and **continuous** variables, which can take infinitely many values within a specified range.

Variables can also be classified in terms of their relative informativeness:[1]

Nominal

These variables comprise a limited number of discrete categories which **cannot be ordered** in a meaningful way. For instance, the genitive forms *of* and *'s* cannot be ordered like numbers.

Ordinal

Ordinal variables are **ordered**. However, the intervals between their individual values are not interpretable. Heumann (2022: 6) provides a pertinent example:

> [T]he satisfaction with a product (unsatisfied–satisfied–very satisfied) is an ordinal variable because the values this variable can take can be ordered but the differences between 'unsatisfied–satisfied' and 'satisfied–very satisfied' cannot be compared in a numerical way.

Interval-scaled

- In the case of **interval**-scaled variables (continuous), the differences between the values can be interpreted, but their ratios must be treated with caution. A temperature of 4°C is 6 degrees warmer than -2°C; however, this does not imply that 4°C is three times warmer than -2°C. This is because the temperature scale has no true zero point; 0°C simply signifies another point on the scale and not the absence of temperature altogether.

Ratio-scaled

- **Ratio**-scaled variables allow both a meaningful interpretation of the differences between their values and (!) of the ratios between them. Within the context of clause length, values such as 4 and 8 not only suggest that the latter is four units greater than the former but also that their ratio $\frac{8}{4} = 2$ is a valid way to describe the relationship between these values. Here a length value of 0 can be clearly viewed as the absence of length altogether.

[1] Note that discrete nominal and ordinal variables are also referred to as **categorical variables**.

Dependent vs. independent variables

In empirical studies, it is often of interest whether one variable is linked to changes in the values of another variable. When exploring such **associations** (or **correlations**), we need to take another heuristic step to clarify the direction of the influence.

In a linguistic context, we denote the variable whose usage patterns we'd like to explain as the **dependent** or **response** variable. A list of possible dependent variables is provided in the section on Linguistic variables.

Its outcomes are said to depend on one or more **independent variables**. These are also often referred to as **explanatory** variables as they are supposed to explain variation in the response variable. In (socio-)linguistics, these include but are not limited to a speaker's age, gender, socio-economic class (e.g., working class vs. middle class), their local variety, or the register of the speech context (e.g., casual exchange vs. business transaction).

Loading datasets into R

We will need `readxl` and `writexl` to aid us with importing and exporting MS Excel files.

```
library(readxl)
library(writexl)
```

If you haven't installed these libraries yet, the R console will throw an error message. For instructions on how to install an R package, consult the unit on Libraries.

Exporting data

Suppose we'd like to export our data frame with word frequencies to a local file on our system. Let's briefly regenerate the data frame:

```
# Generate data frame
data <- data.frame(lemma = c("start", "enjoy", "begin", "help"),
                   frequency = c(418, 139, 337, 281))

# Print contents
print(data)
```

```
  lemma frequency
1 start       418
2 enjoy       139
3 begin       337
4  help       281
```

There are two common formats in which tabular data can be stored:

- as .**csv**-files ('**c**omma-**s**eparated **v**alues'; native format of LibreOffice Calc, for example)
- as .**xls**/.**xlsx**-files (Microsoft Excel)

Export to CSV

To save our `data` data frame in .csv-format, we can use the `write.csv()` function:

```
write.csv(data, "frequency_data.csv")
```

The file is now stored at the location of your current R script. You can open this file …

- in **LibreOffice**
- in **Microsoft Excel** via `File` > `Import` > `CSV file` > Select the file > `Delimited` and then `Next` > `Comma` and `Next` > `General` and `Finish`.

Clearly, opening CSV files in MS Excel is quite cumbersome, which is why it's better to export it as an Excel file directly.

Export to Excel

We use the `write_xlsx()` function provided by the package `writexl`.

```
write_xlsx(data, "frequency_data.xlsx")
```

The file is now stored at the location of your currently active R script. You should now be able to open it in MS Excel without any issues.

Importing data

Let's read the two files back into R.

Import from CSV

To import the CSV file, we can use the `read.csv()` function:

```
imported_csv <- read.csv("frequency_data.csv")
print(imported_csv)
```

```
  X lemma frequency
```

```
1 1 start       418
2 2 enjoy       139
3 3 begin       337
4 4  help       281
```

It appears that `read.csv()` has also written the row numbers to the file. This is not the desired outcome and can be prevented by adding an additional argument:

```r
imported_csv <- read.csv("frequency_data.csv", row.names = 1)
print(imported_csv) # Problem solved!
```

```
  lemma frequency
1 start       418
2 enjoy       139
3 begin       337
4  help       281
```

A note on file encodings and separators

When working with CSV files, you may encounter issues with character encodings and separators, especially when:

- working with files from different operating systems,
- dealing with text containing special characters (é, ü, ñ, etc.), or
- importing files created in different regions (e.g., European vs. US).

The most common encoding-related parameters for read.csv() are:

```r
# For files with special characters (recommended default)
data <- read.csv("myfile.csv", encoding = "UTF-8")

# For files from Windows systems
data <- read.csv("myfile.csv", encoding = "latin1")

# For files using semicolons and commas as decimal points
data <- read.csv("myfile.csv", sep = ";", dec = ",")
```

- If you see garbled text like Ã© instead of é, try specifying `encoding = "UTF-8"`.
- If your data appears in a single column, check if your file uses semicolons (`;`) instead of commas (`,`) as separators.
- If numeric values are incorrect, verify whether the file uses commas or periods as decimal separators.

Import from Excel

For importing the Excel file, we'll use the `read_xlsx()` function from the `readxl` package:

```
imported_excel <- read_xlsx("frequency_data.xlsx")
print(imported_excel)
```

```
# A tibble: 4 x 2
  lemma frequency
  <chr>     <dbl>
1 start       418
2 enjoy       139
3 begin       337
4 help        281
```

That's it! Nevertheless, remember to always check your imported data to ensure it has been read in correctly, especially when working with CSV files.

A convenient alternative: RDS files

If the main goal is to save an intermediary result and make it available for later use, the most efficient solution is to save the object to a local R data file ending in `.RDS`. Since it compressed data, .RDS files can be considered analogous to .zip files, which are very commonly used for other data types.

In practice, we use the `saveRDS()` function and supply it with …

- … an R object (e.g., a vector, data frame, matrix, graphs, statistical models – anything goes!) as well as
- … the desired name of the file.

```
# Save data frame "data" to the file "frequency_data.RDS"
saveRDS(data, "frequency_data.RDS")
```

To read a file back in, we need to indicate the file name (or the full file path if the file is located in a different folder).

```
# Read in "frequency_data.RDS" and assign the contents to "data2"
data2 <- readRDS("frequency_data.RDS")

# Verify contents
print(data2)
```

```
  lemma frequency
1 start       418
2 enjoy       139
3 begin       337
4  help       281
```

Exercises

> **Solutions**
>
> You can find the solutions to the exercises here.

Tier 1

Exercise 1. Consider the following statement:

> This paper examines the influence of clause length on the ordering of main and subordinate clauses.

1. What is the dependent variable?
2. What is the independent variable?
3. What would it mean if they were reversed?

Exercise 2. For each of the following linguistic phenomena, identify whether it is discrete or continuous and then specify the type (nominal, ordinal, interval, or ratio scale):

a. Particle placement (*Cut off the flowers* vs. *Cut the flowers off*)
b. Number of subordinate clauses in a paragraph
c. Speaker's country of origin
d. Reaction time in a language processing task (measured in milliseconds)
e. Likert scale rating of grammaticality (possible ratings: 1, 2, 3, 4, 5)

Exercise 3. Download the dataset `Paquot_Larsson_2020_data.xlsx` from the supplementary materials and store it in a folder you can easily access from within RStudio. [2]

Load the .xlsx file into R and name it `cl.order`. Identify the variable types (nominal, ordinal etc.) for all columns in the `cl.order` dataset.

> **What's in the file? The `str()` function**
>
> The easiest way to get a general overview of the full data set is to apply the `str()` function to the respective data frame.

[2] The dataset was published as part of Paquot & Larsson (2020).

```
str(cl.order)
```

```
tibble [403 x 8] (S3: tbl_df/tbl/data.frame)
 $ CASE       : num [1:403] 4777 1698 953 1681 4055 ...
 $ ORDER      : chr [1:403] "sc-mc" "mc-sc" "sc-mc" "mc-sc" ...
 $ SUBORDTYPE : chr [1:403] "temp" "temp" "temp" "temp" ...
 $ LEN_MC     : num [1:403] 4 7 12 6 9 9 9 4 6 4 ...
 $ LEN_SC     : num [1:403] 10 6 7 15 5 5 12 2 24 11 ...
 $ LENGTH_DIFF: num [1:403] -6 1 5 -9 4 4 -3 2 -18 -7 ...
 $ CONJ       : chr [1:403] "als/when" "als/when" "als/when" "als/when" ...
 $ MORETHAN2CL: chr [1:403] "no" "no" "yes" "no" ...
```

This shows us that the data frame has 8 columns, as the `$` operators indicate (`$ Case`, `$ ORDER`, …). The column names are followed by

- the data type (`num` for numeric and `chr` for character strings)
- the number of values (`` `[1:403]` ``) and
- the first few observations.

Another intuitive way to display the structure of a data matrix is to simply show the first few rows:

```
head(cl.order)
```

```
# A tibble: 6 x 8
   CASE ORDER SUBORDTYPE LEN_MC LEN_SC LENGTH_DIFF CONJ     MORETHAN2CL
  <dbl> <chr> <chr>       <dbl>  <dbl>       <dbl> <chr>    <chr>
1  4777 sc-mc temp            4     10          -6 als/when no
2  1698 mc-sc temp            7      6           1 als/when no
3   953 sc-mc temp           12      7           5 als/when yes
4  1681 mc-sc temp            6     15          -9 als/when no
5  4055 sc-mc temp            9      5           4 als/when yes
6   967 sc-mc temp            9      5           4 als/when yes
```

Further details on the variables

- `ORDER`: Does the subordinate clause come before or after the main clause?
- `SUBORDTYPE`: Is the subordinate clause temporal or causal?
- `MORETHAN2CL`: Are there more clauses in the sentence than just one subordinate clause and one main clause?

- `LEN_MC`: How many words does the main clause contain?
- `LEN_SC`: How many words does the subordinate clause contain?
- `LENGTH_DIFF`: What is the length difference in words between the main clause and subordinate clause?

Number of (unique) values in R

To count the number of items in a vector, which correspond to the total number n of attested values of X, we can use `length()`:

```
length(cl.order$SUBORDTYPE)
```

```
[1] 403
```

In fact, it is equivalent to the number of rows in the full data frame:

```
nrow(cl.order)
```

```
[1] 403
```

The function `unique()` shows all unique items (= "types") in a vector, reflecting the possible outcomes for a single observation:

```
unique(cl.order$SUBORDTYPE)
```

```
[1] "temp" "caus"
```

Tier 2

Exercise 4. Explain why frequency data is discrete. What is their level of measurement (nominal, ordinal, interval-scale, ratio-scale)? Give an example to justify your answer.

Exercise 5. Download the file `SCOPE_reduced.RDS` from this repository and read it into a variable named `SCOPE`. It contains data from the the South Carolina Psycholinguistic Metabase (Gao, Shinkareva, and Desai 2022), specifically:

- Number of meanings (`Nsenses_WordNet`)
- Emotional valence ratings, which describe the pleasantness of a lexical stimulus on a scale from 1 to 9 (`Valence_Warr`)
- Data for nearly 200,000 words, but there are also many missing values (signified by `NA`)

a. To familiarise yourself with the dataset, use subsetting operations from Base R or the `tidyverse` to retrieve from the `SCOPE` data frame:

   1. the number of meanings for the verbs *start*, *enjoy*, *begin*, *help*. Store them in a data frame with the name `senses_df`.
   2. emotional valence ratings for the words *fun*, *love*, *vacation*, *war*, *politics*, *failure*, *table*. Store them in a data frame, and name it `valence_df`.

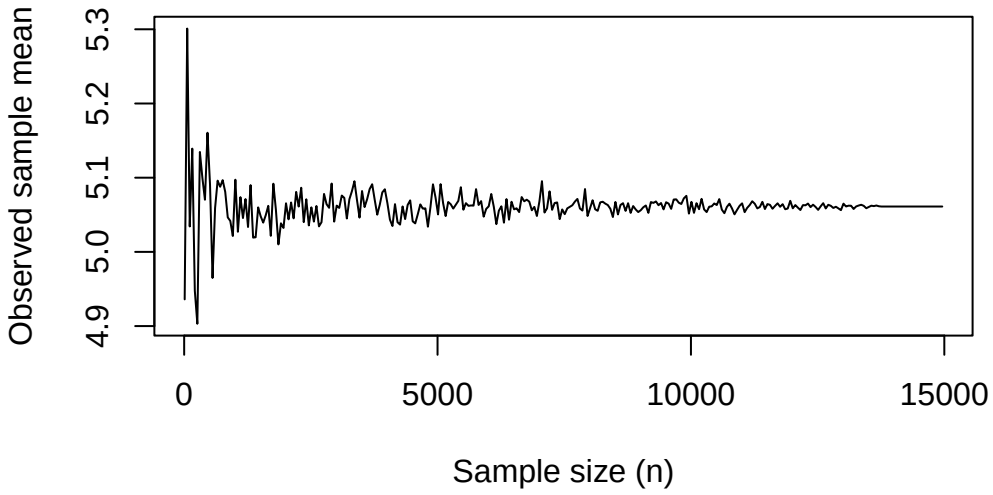
   What do you notice about the valence ratings? Do they align with your intuitions about these words' emotional content?

b. Consider the following simulation study. Let μ_{Valence} denote the average valence rating in the population. We will repeatedly sample words from `SCOPE` for varying values of n and compute the average ratings $\hat{\mu}_{\text{Valence}}$.

```r
# Define a range of sample sizes to try
sample_sizes <- seq(10, 15000, by = 50)

# For each sample size, draw a random sample and compute x_bar
set.seed(123)
x_bars <- sapply(sample_sizes, function(n) {
  sample_data <- SCOPE %>%
    filter(!is.na(Valence_Warr)) %>%
    slice_sample(n = n, replace = FALSE)  # without replacement
  mean(sample_data$Valence_Warr)
})

# Plot the results
plot(sample_sizes, x_bars,
     type = "l",
     xlab = "Sample size (n)",
     ylab = "Observed sample mean")
```



As n increases, $\hat{\mu}_{\text{Valence}}$ seems to stabilise around a fixed value. What does the convergence tell us about the relationship between sample statistics and population parameters? What does this suggest about the minimum sample size a researcher would need to obtain a reliable estimate of $\hat{\mu}_{\text{Valence}}$?

Tier 3

Exercise 6. The `tidyverse` function `slice_sample()` allows users to randomly sample rows from a data frame, with every row having the same probability of being selected. We will do so without replacement for the `cl.order` data, choosing 50 rows at random.

```
library(tidyverse)

# For reproducibility, it is highly recommended to set a random seed. This
 ↪  makes sure we will get the "same" random result every time we execute
 ↪  our code.
set.seed(123)

# Extract a random sample of 50 rows
cl.order.random <- cl.order %>%
  slice_sample(n = 50, replace = FALSE)

print(cl.order.random)
```

```
# A tibble: 50 x 8
    CASE ORDER SUBORDTYPE LEN_MC LEN_SC LENGTH_DIFF CONJ          MORETHAN2CL
   <dbl> <chr> <chr>       <dbl>  <dbl>       <dbl> <chr>         <chr>
 1  3490 sc-mc temp           15     10           5 nachdem/after no
 2  4608 mc-sc temp            5     11          -6 als/when      yes
 3  2458 sc-mc temp           29      4          25 nachdem/after no
 4  3178 sc-mc caus            4     19         -15 weil/because  yes
 5  3616 sc-mc temp            6      5           1 bevor/before  yes
 6  2405 mc-sc caus            6      5           1 weil/because  no
 7  1799 mc-sc caus            3      7          -4 weil/because  no
 8  2752 mc-sc caus           13     16          -3 weil/because  no
 9   350 mc-sc caus            8     11          -3 weil/because  no
10   342 mc-sc caus           10     15          -5 weil/because  no
# i 40 more rows
```

Another common approach is **stratified sampling**, which is a method of sampling from a population where the population is divided into distinct subgroups (strata) based on a characteristic, and a sample is drawn from each subgroup to ensure proportional representation.

```r
library(sampling)
library(tidyverse)

strat_sample <- function(data, variable, size) {

  # Capture the variable symbol
  var_sym <- enquo(variable)
  var_name <- rlang::as_name(var_sym)

  # Proportions in the population (= sampling weights)
  data_prop <- table(pull(data, !!var_sym)) / nrow(data)

  # Sizes of the stratified sample
  strat_sample_sizes <- round(size * data_prop)

  # Convert variable of interest to factor
  data[[var_name]] <- as.factor(data[[var_name]])

  # Draw the sample
  clause_strat_sample <- strata(data,
                                stratanames = var_name,
                                size = strat_sample_sizes,
                                # Stratified sampling without replacement
                                method = "srswor")

  # Output df
  output_sample <- tibble(getdata(data, clause_strat_sample)) %>%
    select(-Prob, -Stratum, -ID_unit)

  return(output_sample)
}

# Apply the function
cl.order.strat <- strat_sample(cl.order, ORDER, size = 50)
```

Let's compare the results of the different sampling procedures:

```r
# Check proportions in the original dataset
cl.order %>%
  count(ORDER) %>%
  mutate(pct = n/sum(n))
```

```
# A tibble: 2 x 3
  ORDER     n   pct
```

```
  <chr> <int> <dbl>
1 mc-sc   275 0.682
2 sc-mc   128 0.318
```

```
# Check proportions in the random sample
cl.order.random %>%
   count(ORDER) %>%
   mutate(pct = n/sum(n))
```

```
# A tibble: 2 x 3
  ORDER     n   pct
  <chr> <int> <dbl>
1 mc-sc    29  0.58
2 sc-mc    21  0.42
```

```
# Check proportions in the stratified sample
cl.order.strat %>%
   count(ORDER) %>%
   mutate(pct = n/sum(n))
```

```
# A tibble: 2 x 3
  ORDER     n   pct
  <fct> <int> <dbl>
1 mc-sc    16  0.32
2 sc-mc    34  0.68
```

a. Perform both random and stratified sampling based on the `CONJ` column. For each conjunction type, compute the absolute difference between its proportion in the random sample and its proportion in the stratified sample. Which sampling method better preserves the distribution of `CONJ` found in the full dataset, and why?
b. Based on your findings in (a), discuss when stratified sampling would be preferable over random sampling in linguistic research. Under what circumstances might random sampling be sufficient?

Exercise 7. Assuming three randomly chosen observations from the `cl.order` dataset, describe the sample space S for the variable `ORDER`. Define at least two random variables with domain (= input space) S, specifying for each what it counts or measures.

References

Agresti, Alan, and Maria Kateri. 2022. *Foundations of Statistics for Data Scientists: With r and Python*. Boca Raton: CRC Press.

Baayen, R. Harald. 2008. *Analyzing Linguistic Data: A Practical Introduction to Statistics Using r*. Camrbidge: Cambridge University Press.

Baguley, Thomas. 2012. *Serious Stats: A Guide to Advanced Statistics for the Behavioral Sciences*. Houndmills, Basingstoke: Palgrave Macmillan.

Gao, Chuanji, Svetlana V. Shinkareva, and Rutvik H. Desai. 2022. "SCOPE: The South Carolina Psycholinguistic Metabase." *Behavior Research Methods* 55 (6): 2853–84. https://doi.org/10.3758/s13428-022-01934-0.

Heumann, Christian, Michael Schomaker, and Shalabh. 2022. *Introduction to Statistics and Data Analysis: With Exercises, Solutions and Applications in r*. 2nd ed. Cham: Springer. https://doi.org/10.1007/978-3-031-11833-3.

Paquot, Magali, and Tove Larsson. 2020. "Descriptive Statistics and Visualization with r." In *A Practical Handbook of Corpus Linguistics*, edited by Magali Paquot and Stefan Thomas Gries, 375–99. Cham: Springer.